МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ

УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ
«БРЕСТСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ

ФАКУЛЬТЕТ ЭЛЕКТРОННО-ИНФОРМАЦИОННЫХ СИСТЕМ

Кафедра интеллектуальных информационных технологий

Отчет по лабораторной работе №2

Специальность СУИ

Выполнил:
Кульбеда А. Ю.
студент группы АС-68

Проверил:
Крощенко А. А.
ст. преп. кафедры ИИТ,

«___» _____ 2026г.

Брест 2026

Цель работы: изучить применение линейной и логистической регрессии для решения практических задач. Научиться обучать модели, оценивать их качество с помощью соответствующих метрик и интерпретировать результаты.

Вариант 4

Задание 1. Регрессия (Определение веса рыбы)

1. Fish Market
2. Предсказать вес рыбы (Weight)
3. Задания:
    - Загрузите данные. В качестве признаков используйте Length1, Length2, Length3, Height, Width;
    - Обучите модель линейной регрессии;
    - Оцените качество, рассчитав R^2 и RMSE (Root Mean Squared Error);
    - Постройте диаграмму рассеяния для Length3 и Weight с линией регрессии.

Код программы:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

# загружаем данные
df = pd.read_csv('Fish.csv')

# признаки и цель
X = df[['Length1', 'Length2', 'Length3', 'Height', 'Width']]
y = df['Weight']

# делим на train/test
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)

# обучаем линейную регрессию
model = LinearRegression()
model.fit(X_train, y_train)

# предсказания и метрики
pred = model.predict(X_test)
rmse = np.sqrt(mean_squared_error(y_test, pred))
r2 = r2_score(y_test, pred)

print("R2 =", r2)
print("RMSE =", rmse)

# график Length3 vs Weight с линией регрессии
```

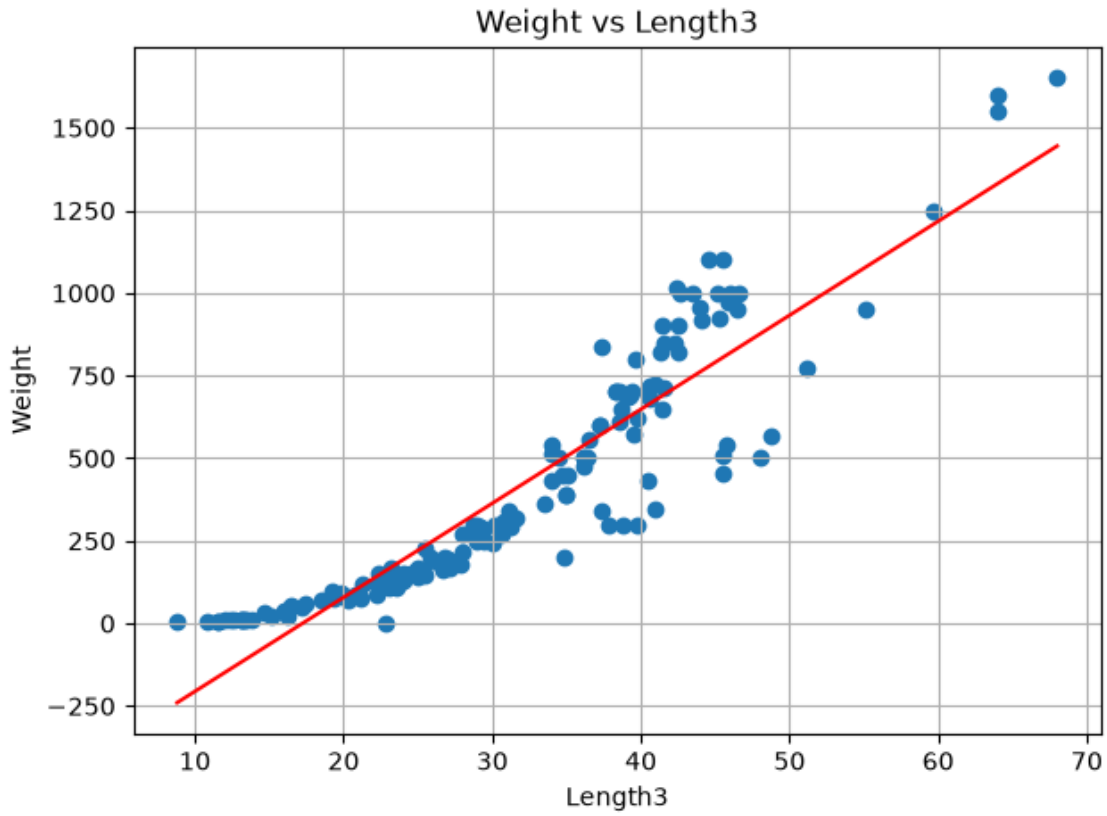
```
plt.figure(figsize=(7,5))
plt.scatter(df['Length3'], df['Weight'])

# отдельная модель только по Length3, чтобы линия была ровной
m2 = LinearRegression()
m2.fit(df[['Length3']], df['Weight'])
x_line = np.linspace(df['Length3'].min(), df['Length3'].max(),
100).reshape(-1,1)
y_line = m2.predict(x_line)
plt.plot(x_line, y_line, color='red')

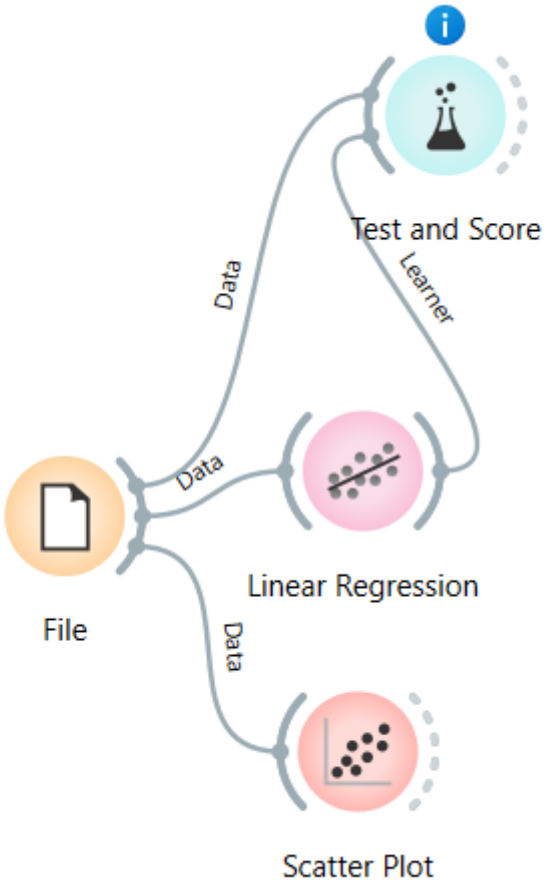
plt.xlabel('Length3')
plt.ylabel('Weight')
plt.title('Weight vs Length3')
plt.grid(True)
plt.show()
```

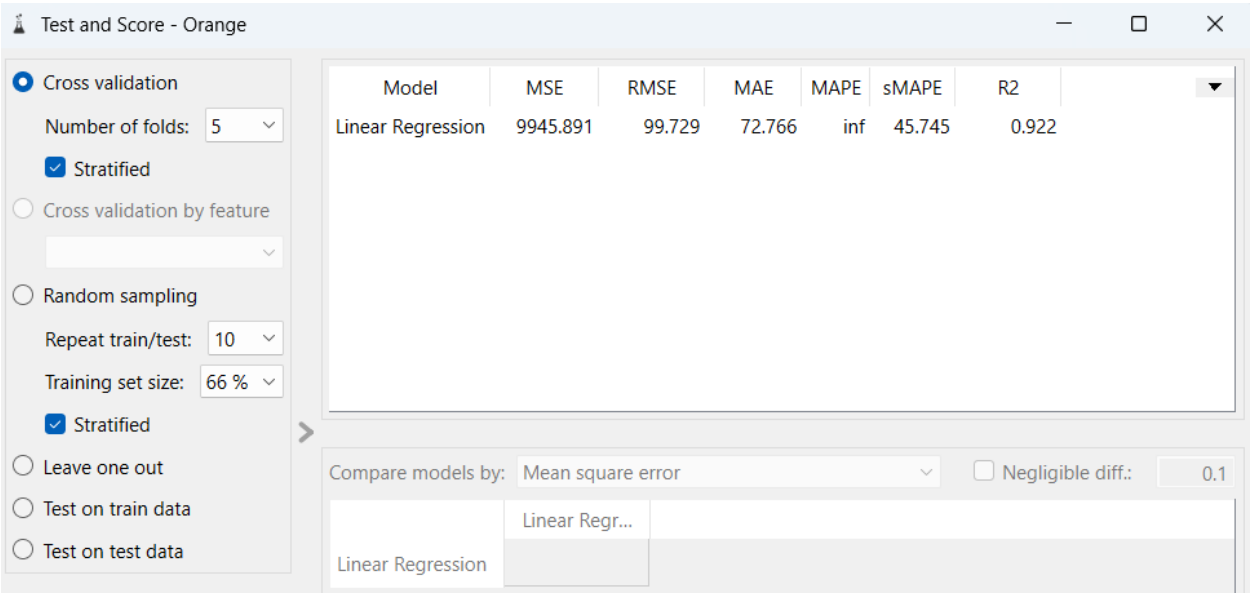
Вывод программы:

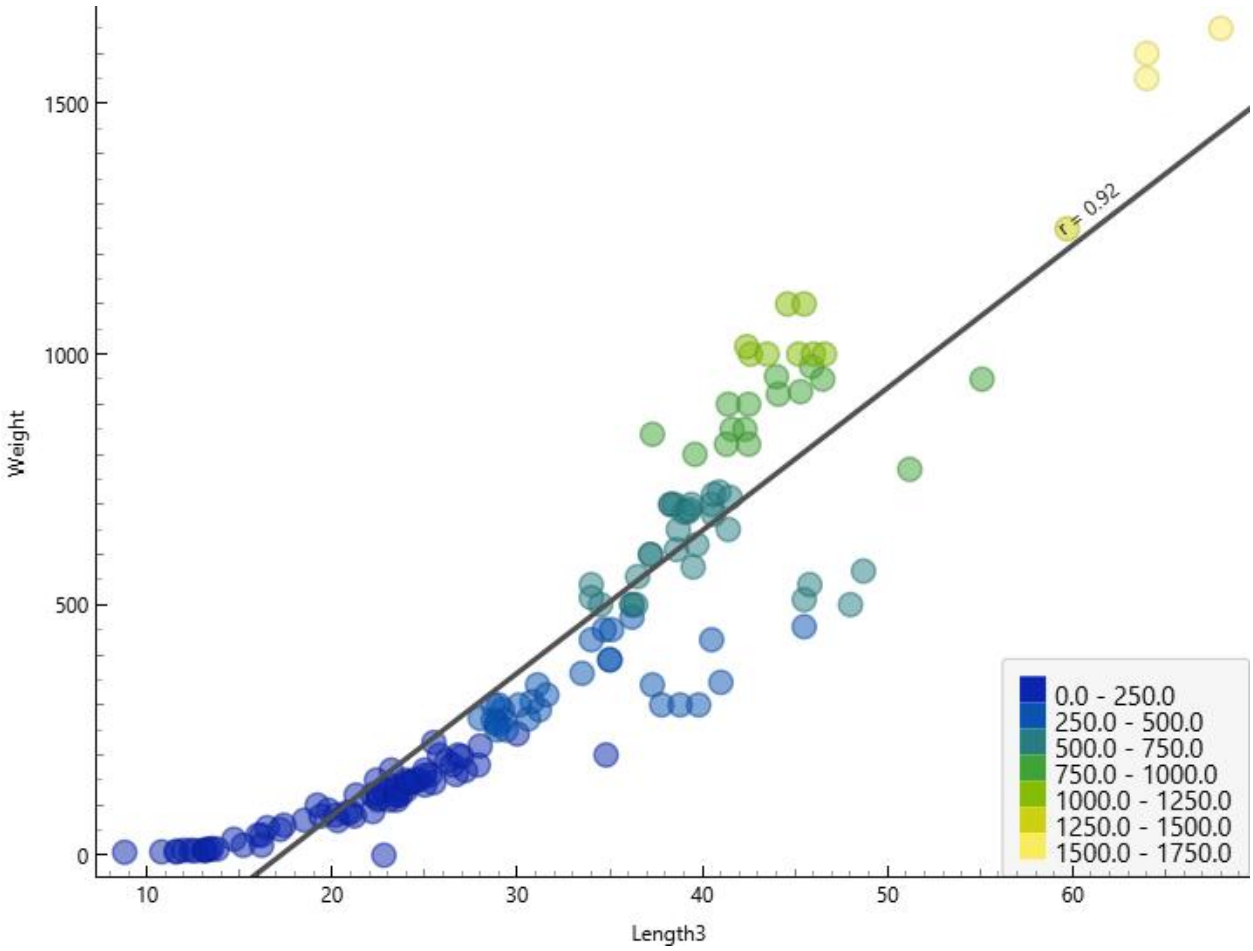
```
R2 = 0.8821430593048696
RMSE = 129.47543084748062
C:\Users\nkola\AppData\Local\Python\pythoncore-3.15-64\Lib\site-packages\sklearn\utils\validation.py:2830:
UserWarning: X does not have valid feature names, but LinearRegression was fitted with feature names
  warnings.warn(
```



Результаты программы Orange:







Вывод по заданию 1:

Модель хорошо справилась с задачей. В коде R^2 получился 0.882, а в Orange — 0.922. Это не ошибка, просто в коде данные делились один раз (80% на обучение, 20% на проверку), а в Orange проверка идет в 5 этапов (кросс-валидация) на всех данных. Из-за этого оценка в Orange получилась чуть точнее и стабильнее. Еще Orange сам автоматически добавил вид рыбы в обучение, поэтому результат немного выше.

Задание 2. Классификация (Прогнозирование отклика на банковское предложение)

1. Bank Marketing UCI
2. Предсказать, подпишется ли клиент на срочный вклад (y)
3. Задания:
    - Загрузите данные, преобразуйте категориальные признаки;
    - Обучите модель логической регрессии;
    - Рассчитайте Accuracy, Precision и Recall для класса “yes”;
    - Постройте матрицу ошибок.

Код программы:

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, precision_score,
recall_score, confusion_matrix, ConfusionMatrixDisplay

# читаем файл, разделитель ;
data = pd.read_csv('bank.csv', sep=';')

# y -> 1/0
data['y'] = data['y'].map({'yes': 1, 'no': 0})

# категориальные признаки кодируем one-hot
X = pd.get_dummies(data.drop('y', axis=1), drop_first=True)
y = data['y']

# train/test
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# логистическая регрессия
log = LogisticRegression(max_iter=1000)
log.fit(X_train, y_train)

# предсказания
pred = log.predict(X_test)

# метрики
acc = accuracy_score(y_test, pred)
prec = precision_score(y_test, pred)
rec = recall_score(y_test, pred)

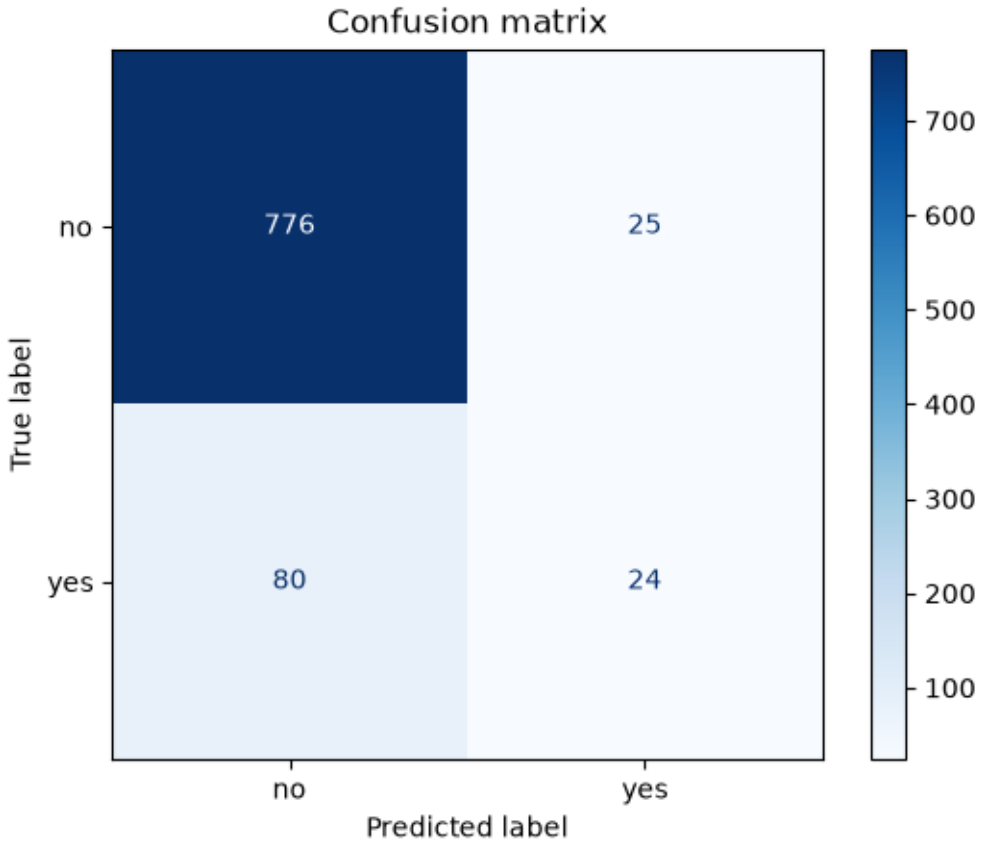
print("Accuracy =", acc)
print("Precision =", prec)
print("Recall =", rec)

# матрица ошибок
cm = confusion_matrix(y_test, pred)
disp = ConfusionMatrixDisplay(confusion_matrix=cm,
display_labels=['no', 'yes'])
disp.plot(cmap='Blues')
plt.title('Confusion matrix')
plt.show()
```

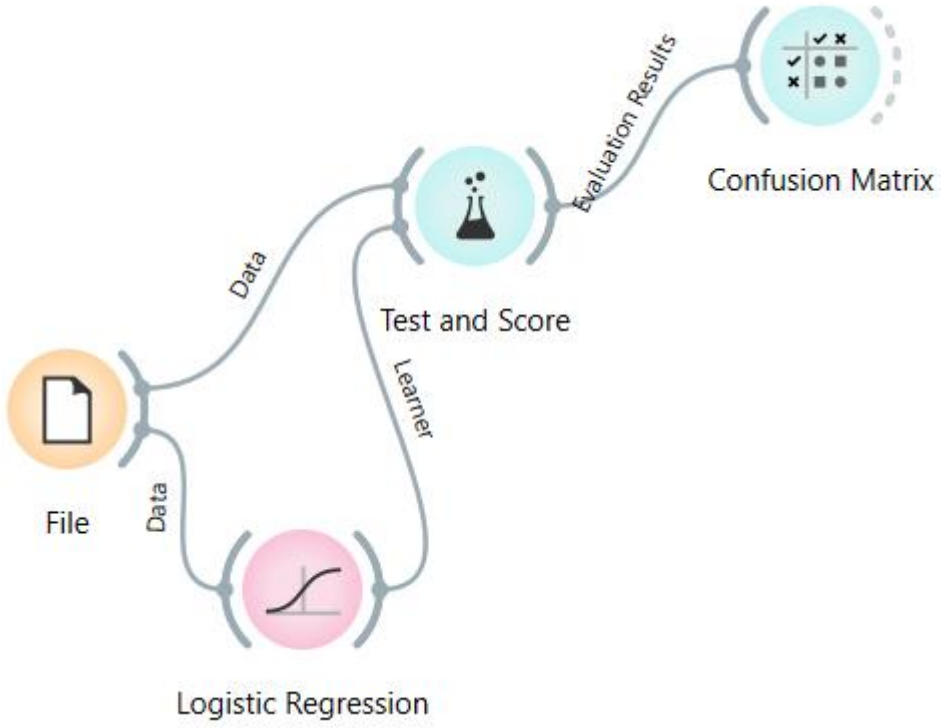
Вывод программы:

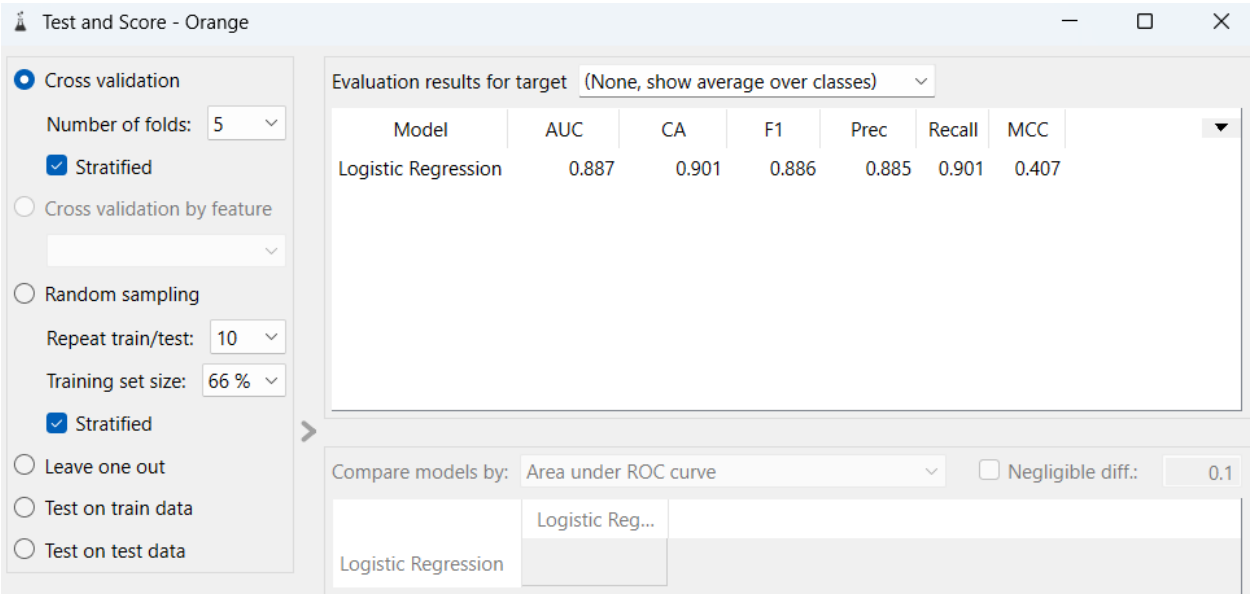
```
599: ConvergenceWarning: lbfgs failed to converge after 1000 iteration(s) (status=1):
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT

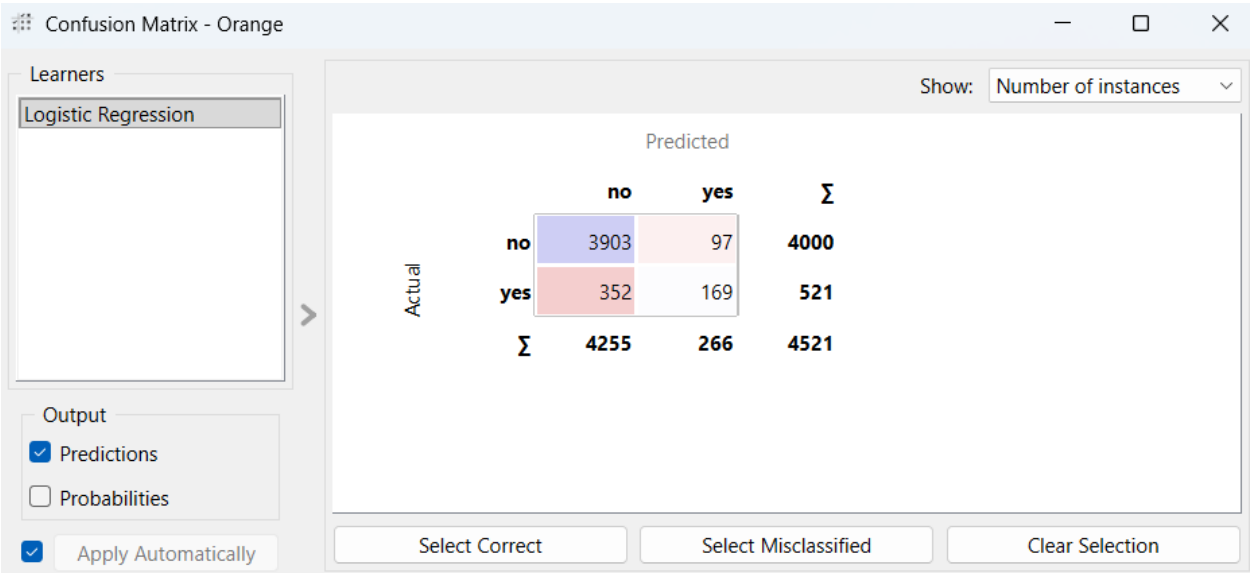
Increase the number of iterations to improve the convergence (max_iter=1000).
You might also want to scale the data as shown in:
    https://scikit-learn.org/stable/modules/preprocessing.html
Please also refer to the documentation for alternative solver options:
    https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression
  n_iter_i = _check_optimize_result(
Accuracy = 0.8839779005524862
Precision = 0.4897959183673469
Recall = 0.23076923076923078
```



Результаты программы Orange:







Вывод по заданию 2:

Модель предсказывает отказы клиентов очень хорошо, а вот согласия («yes») находит плохо. Если сравнить код и Orange, там разные цифры, потому что в коде тестировалось только 20% данных, а в Orange — все 100% по очереди. В итоге модель редко ошибается в целом (90% правильных ответов), но из тех, кто реально хотел оформить вклад, она находит только треть. Для банка это плохо, потому что они теряют потенциальных клиентов.